

UNIVERSIDAD RAFAEL LANDÍVAR

FACULTAD DE INGENIERÍA

LICENCIATURA EN INGENIERIA EN INFORMATICA Y SISTEMAS

ESTRUCUTURAS DE DATOS II

PROYECTO 1 (Manual Tecnico)

NOMBRE:

Marvin Antonio Vásquez Chán

CARNET:

1558525

QUETZALTENANGO, SEPTIEMBRE DE 2026

CAMPUS SAN ALBERTO HURTADO, S. J

MANUAL TÉCNICO

Futstats MV – Estadísticas del Mundial

Proyecto de Estructuras de Datos II
Aplicación de escritorio desarrollada en C# y Windows Forms

Sistema	Futstats MV – Estadísticas del Mundial
Tecnología	C# / .NET 8 / Windows Forms
Persistencia	Archivos CSV
Estructuras principales	Árbol B+, Max Heap y Min Heap

1. Introducción

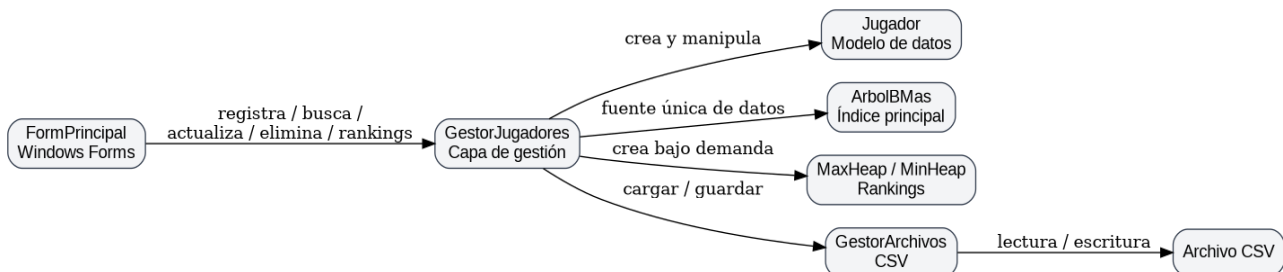
Este manual técnico describe la estructura, funcionamiento y componentes del sistema Futstats MV, una aplicación de escritorio orientada a la gestión y consulta de estadísticas de jugadores de un Mundial. El sistema fue diseñado para demostrar el uso práctico de estructuras de datos dinámicas, especialmente un Árbol B+ como índice principal y heaps para generar rankings.

2. Objetivo del sistema

Permitir registrar, consultar, actualizar y eliminar jugadores, visualizar un catálogo ordenado por Id, importar y exportar información mediante CSV y generar rankings por goles, asistencias, minutos jugados, tarjetas, partidos y porterías a cero.

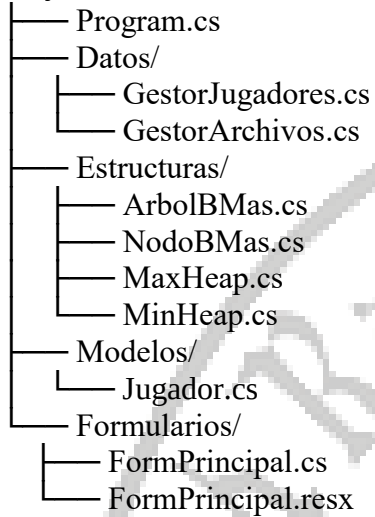
3. Arquitectura del sistema

La aplicación utiliza una separación lógica en cuatro niveles: interfaz gráfica, gestión de datos, estructuras de datos/modelos y manejo de archivos. FormPrincipal no accede directamente al Árbol B+ ni a los heaps; todas las operaciones pasan por GestorJugadores.



4. Estructura del proyecto

ProyectoMundial/



5. Componentes principales

Componente	Responsabilidad
Program.cs	Punto de entrada. Configura DPI, estilos de Windows Forms y ejecuta FormPrincipal.
FormPrincipal.cs	Interfaz gráfica. Contiene las pestañas Registro, Búsqueda, Rankings y Catálogo completo, además de los botones de CSV.
Jugador.cs	Modelo que representa un jugador y sus estadísticas. También convierte registros a/desde líneas CSV.
GestorJugadores.cs	Capa central de negocio. Mantiene el Árbol B+ como fuente única de datos y crea heaps bajo demanda.
ArbolBMas.cs	Índice principal por Id. Es de orden 4, admite inserción, búsqueda, eliminación y recorrido ordenado.
NodoBMas.cs	Define la estructura de cada nodo del Árbol B+: claves, datos, hijos y enlace a la siguiente hoja.

MaxHeap.cs	Heap máximo usado para obtener listados completos ordenados de mayor a menor.
MinHeap.cs	Heap mínimo acotado usado para calcular Top N de forma eficiente.
GestorArchivos.cs	Responsable exclusivamente de lectura y escritura de archivos CSV.

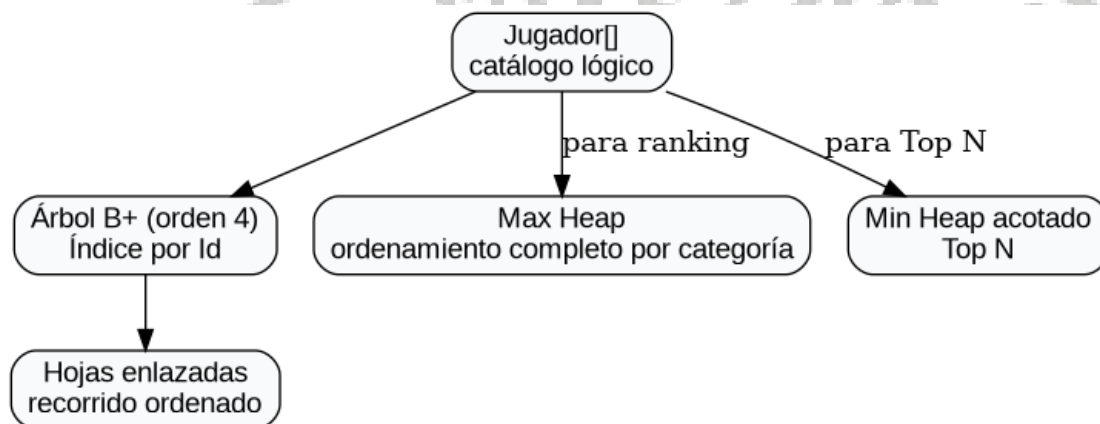
6. Modelo de datos: Jugador

Cada jugador contiene los siguientes campos:

Campo	Tipo	Descripción
Id	string	Identificador único y clave de búsqueda.
Nombre	string	Nombre del jugador.
Seleccion	string	País o selección a la que pertenece.
Posicion	string	Portero, Defensa, Mediocampista o Delantero.
MinutosJugados	int	Minutos acumulados.
Goles	int	Cantidad de goles.
Asistencias	int	Cantidad de asistencias.
TarjetasAmarillas	int	Tarjetas amarillas.
TarjetasRojas	int	Tarjetas rojas.

PartidosJugados	int	Partidos disputados.
PorteriasACero	int	Porterías a cero, especialmente relevante para porteros.

7. Estructuras de datos



7.1 Árbol B+

El Árbol B+ de orden 4 es la estructura principal del sistema. El Id del jugador se utiliza como clave. Los nodos internos funcionan como guías de búsqueda y los datos reales permanecen en las hojas. Las hojas están enlazadas mediante Siguiente, permitiendo recorrer el catálogo de izquierda a derecha en orden creciente de Id.

Operaciones implementadas:

- Insertar jugador y dividir hojas/nodos cuando se alcanza la capacidad.
- Buscar por Id recorriendo desde la raíz hasta la hoja correspondiente.
- Eliminar por Id y rebalancear mediante préstamo o fusión cuando es necesario.
- Recorrer todas las hojas en orden.

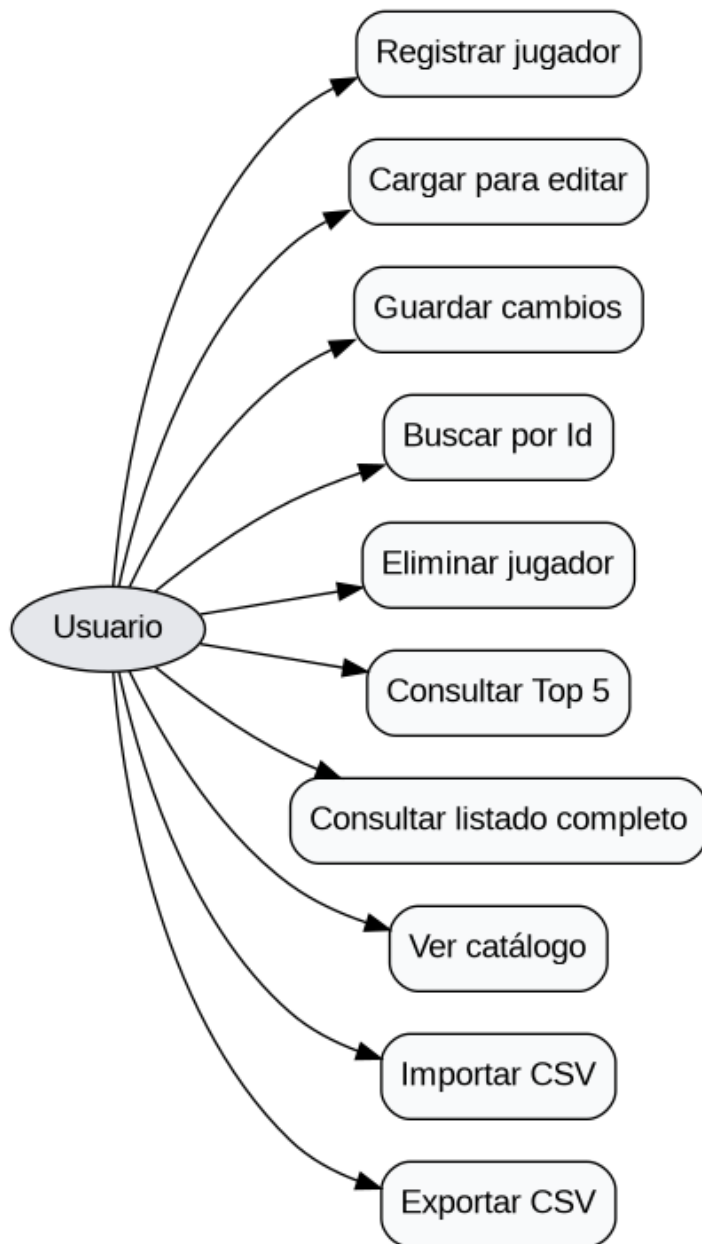
7.2 Max Heap

Se construye temporalmente para rankings completos. La raíz contiene el valor máximo de la categoría seleccionada. Al extraer repetidamente el máximo se obtiene el listado de mayor a menor.

7.3 Min Heap acotado

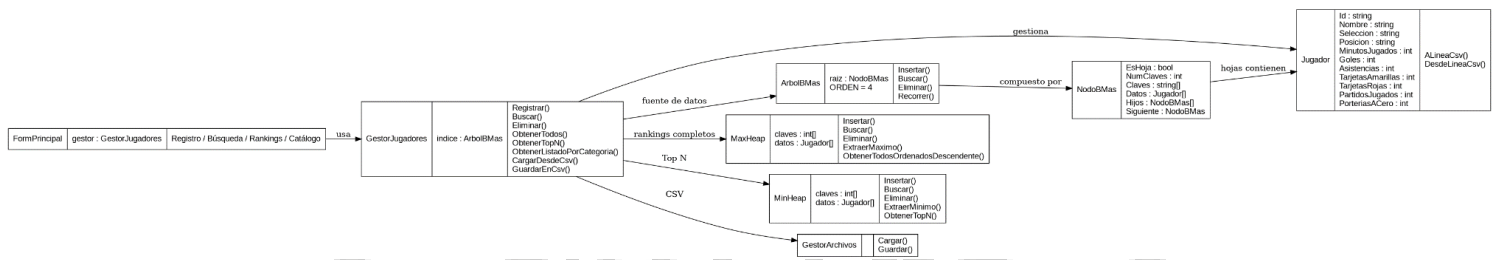
Se utiliza para Top 5. En lugar de ordenar todos los jugadores, mantiene únicamente los mejores 5 candidatos. La raíz representa el menor de los elementos que actualmente forman parte del Top 5; cuando aparece un jugador mejor, se reemplaza ese mínimo.

8. Casos de uso



- Registrar jugador
- Cargar datos para editar y guardar cambios
- Buscar jugador por Id
- Eliminar jugador
- Consultar Top 5 por categoría
- Consultar listado completo por categoría
- Visualizar catálogo completo
- Cargar jugadores desde CSV
- Guardar catálogo en CSV

9. Diagrama de clases



10. Funcionalidades de la interfaz

Módulo	Funcionamiento
Registro	Permite ingresar Id, nombre, selección, posición y estadísticas. Incluye registrar nuevo, cargar para editar y guardar cambios.
Búsqueda	Recibe un Id y muestra todos los datos del jugador encontrado. Desde esta vista también puede eliminarse.
Rankings	Permite seleccionar una categoría y mostrar Top 5 o listado completo.
Catálogo completo	Muestra todos los jugadores ordenados por Id mediante el recorrido de las hojas del Árbol B+.
CSV	Cargar desde CSV agrega únicamente jugadores cuyos Id aún no existen. Guardar en CSV exporta el catálogo actual.

11. Complejidad de las operaciones

Operación	Complejidad aproximada	Observación
Buscar por Id	$O(\log n)$	Árbol B+ de orden fijo.
Insertar	$O(\log n)$	Puede requerir divisiones y propagación hacia la raíz.

Eliminar	$O(\log n)$	Puede requerir préstamo o fusión.
Recorrer catálogo	$O(n)$	Se recorren las hojas enlazadas.
Ranking completo	$O(n \log n)$	Construcción por inserciones en MaxHeap y extracciones.
Top 5	$O(n \log 5) \approx O(n)$	Min Heap acotado; $k=5$.
Buscar dentro de Heap	$O(n)$	Un heap no garantiza orden para búsquedas arbitrarias.
Cargar CSV	$O(n \log n)$	Cada jugador válido se registra en el Árbol B+.
Guardar CSV	$O(n)$	Recorrido y escritura secuencial.

12. Formato CSV

El sistema trabaja con registros separados por comas. El orden esperado de los datos es:

Id,Nombre,Seleccion,Posicion,MinutosJugados,Goles,Asistencias,TarjetasAmarillas,TarjetasRojas,PartidosJugados,PorteriasACero

Nota técnica: la implementación actual de GestorArchivos.Guardar escribe la columna PorteriasACero mediante ALineaCsv(), pero el texto de encabezado de Guardar() no la incluye. Conviene corregir el encabezado para que coincida exactamente con las 11 columnas de datos.

13. Requisitos técnicos

- Sistema operativo Windows.
- .NET 8 con soporte para Windows Forms.
- Visual Studio 2022 o un entorno compatible con .NET 8 y Windows Forms.
- No se utilizan bases de datos externas; la persistencia se realiza mediante archivos CSV.
- El proyecto debe ejecutarse en Windows porque utiliza Windows Forms.

14. Instalación y ejecución

1. Extraer el proyecto.
2. Abrir la solución o el proyecto en Visual Studio.
3. Verificar que el proyecto tenga como destino net8.0-windows.
4. Establecer ProyectoMundial como proyecto de inicio si Visual Studio solicita seleccionar uno.
5. Compilar mediante Compilar > Compilar solución.
6. Ejecutar con F5 o Ctrl+F5.
7. Para cargar datos, utilizar Cargar desde CSV y seleccionar un archivo con el formato indicado.

15. Mantenimiento y recomendaciones técnicas

- Mantener Id como identificador único; no reutilizarlo para representar otro jugador sin una operación de actualización.
- Validar datos numéricos y rangos antes de registrar información si el sistema se amplía.
- Si los nombres o selecciones pueden contener comas, migrar el parser CSV a un lector que soporte campos entrecomillados.
- Corregir el encabezado CSV para incluir PorteriasACero.
- Separar eventualmente la lógica de validación de la interfaz para facilitar pruebas unitarias.
- Agregar pruebas automáticas para inserción, división, eliminación y rebalanceo del Árbol B+.
- Evitar incluir carpetas bin, obj y .vs en entregas finales o repositorios de distribución.

16. Conclusión técnica

Futstats MV presenta una arquitectura adecuada para un proyecto académico de Estructuras de Datos II: la información se centraliza en un Árbol B+, mientras que los heaps se utilizan de manera especializada para resolver rankings. Esta decisión evita utilizar una única estructura para todas las operaciones y permite demostrar ventajas concretas de cada estructura. La interfaz Windows Forms mantiene separada la presentación de la lógica de gestión y del almacenamiento en CSV.

17. Anexo: relación entre clases y responsabilidades

- Program → inicia la aplicación.
- FormPrincipal → interacción con el usuario.
- GestorJugadores → coordina las operaciones sobre jugadores.
- Jugador → entidad de información.
- ArbolBMas / NodoBMas → almacenamiento e indexación por Id.
- MaxHeap → ranking completo.
- MinHeap → Top N.
- GestorArchivos → persistencia CSV.